

学号： 2106410117



沈阳建筑大学

智能手机应用程序开发 综合设计报告

2023 年_秋_季学期

设计题目： 五子棋博弈软件
学 院： 计算机科学与工程学院
专业班级： 计算机 2101 班
姓 名： 杨金娜

问题描述：

课程设计要求：Android 软件一个，设计不少于 5 个页面，如使用数据库则数据库不少于 5 张表，不使用数据库则至少使用一个传感器(摄像头、麦克风、加速度、光线、指纹、GPS 等)，信息本地存储或云存储均可。

软件描述：

这是一个关于五子棋博弈游戏的安卓软件，实现游戏双方的博弈，可以是机器人与真人的博弈，也可以是人与人直接的博弈，还可以是网络上陌生人相互之间的博弈。适应现在网络化需求，要求界面友好，美观，操作方便，充分利用系统现有控件。使用者拥有一个良好的五子棋博弈体验。

设计概要：

五子棋博弈软件是基于安卓系统的一个游戏软件，软件整体上分成几个功能模块：注册模块、登录模块、单人对战模块、双人对战模块、联机对战模块。然后还有五个以上的数据库表格来储存自己的数据。

建立一个完整的五子棋博弈软件

注册模块实现程序的数据插入数据库、登录模块实现调用数据库的数据进行身份核对、单人对战模块实现了由电脑来充当对手与玩家进行博弈。双人对战模块则是为了玩家提供了一个了一个对战平台，实现记录等功能。联机对战模板满足了广大玩家寻找对手的需求，突破了空间的限制。

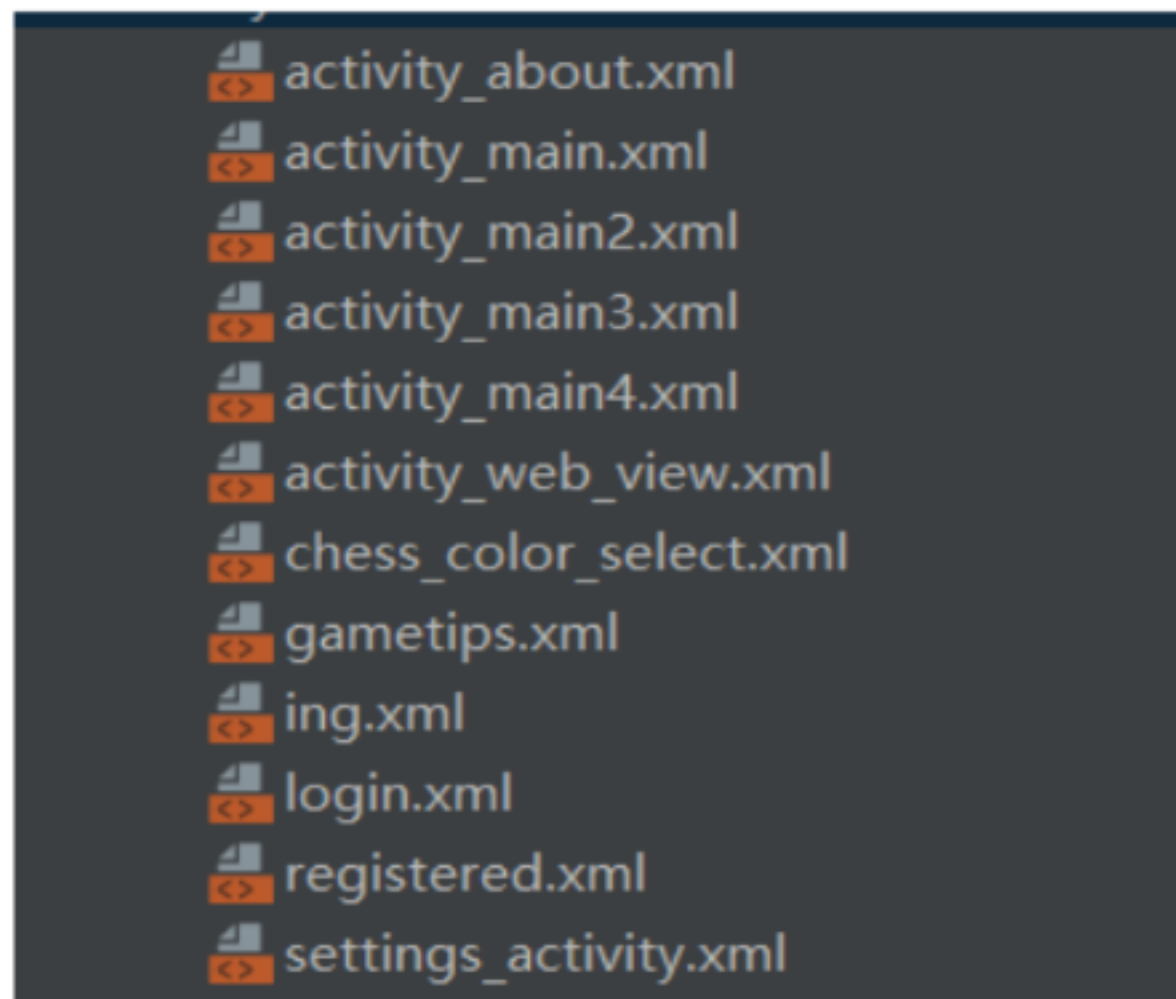
整体逻辑：

使用 JAVA 语言编写后端业务逻辑，使用 SQLite 数据库进行数据存储，编辑前端页面配套后端代码实现相关的功能。并实现服务器线程控制，让玩家提供服务器程序进行数据的交互。在算法设计方面，采用现在主流的博弈算法极大值-极小值搜索算法（alpha-beta 剪枝搜索），设置学习深度为 4，这个深度已经足够应对大部分的玩家而且不会有太多的计算时间。

使用的软件：IntelliJ IDEA 2023.15，MySQL 数据库，Android Studio，Navicat Premium 15。

前端界面：

本次的五子棋博弈软件的前端页面不少于五个界面，如下图所示：



activity_about.xml:关于界面，用于显示游戏版本、制作人员及作者微博。

activity_main.xml:主界面，包括软件的背景登录注册按钮，以及选择游戏模式：单人游戏、双人游戏、联机游戏的按钮；左上角显示登录的用户名。

activity_main2.xml、activity_main3.xml\ activity_main4.xml:单人、双人及联机模式下的游戏棋盘图；悔棋、游戏重开、退出游戏的按钮。

activity_web_view.xml:跳转作者微博的页面。

chess_color_select:选择棋子颜色，黑色先手，白色后手。

gametips.xml:联机对战结束后，弹窗提示选择是否再来一局。

Ing.xml:联机对战匹配对手时，弹窗等待提示及取消按钮。

login.xml:弹窗界面，用于用户输入账号密码登录。

registered.xml:弹窗界面，用于用户注册。

setting_activity.xml:菜单。

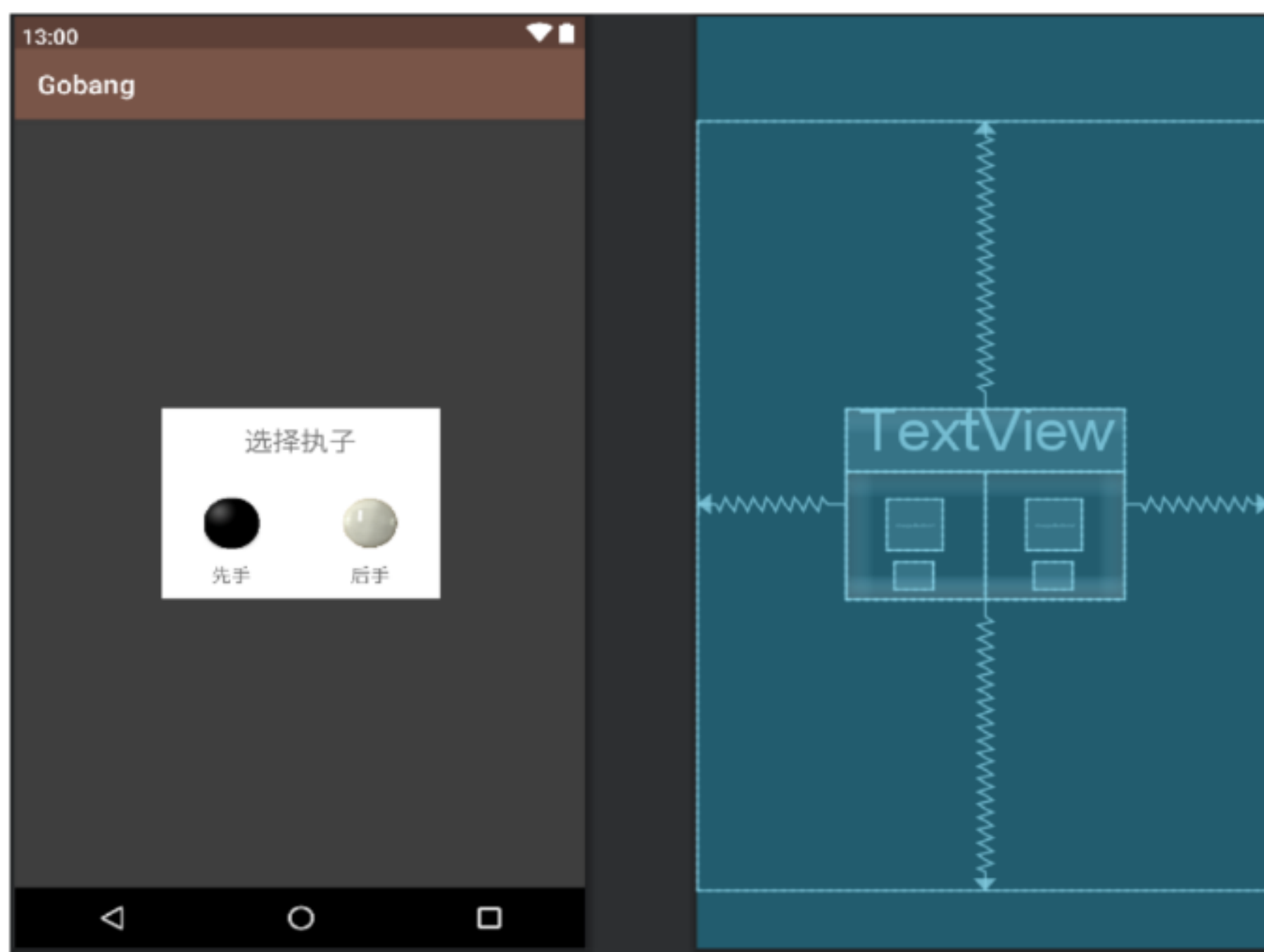
menu_main_activity.xml:菜单界面，包括：登录、注册、设置、关于、退出功能。

接下来是我的部分前端页面展示：

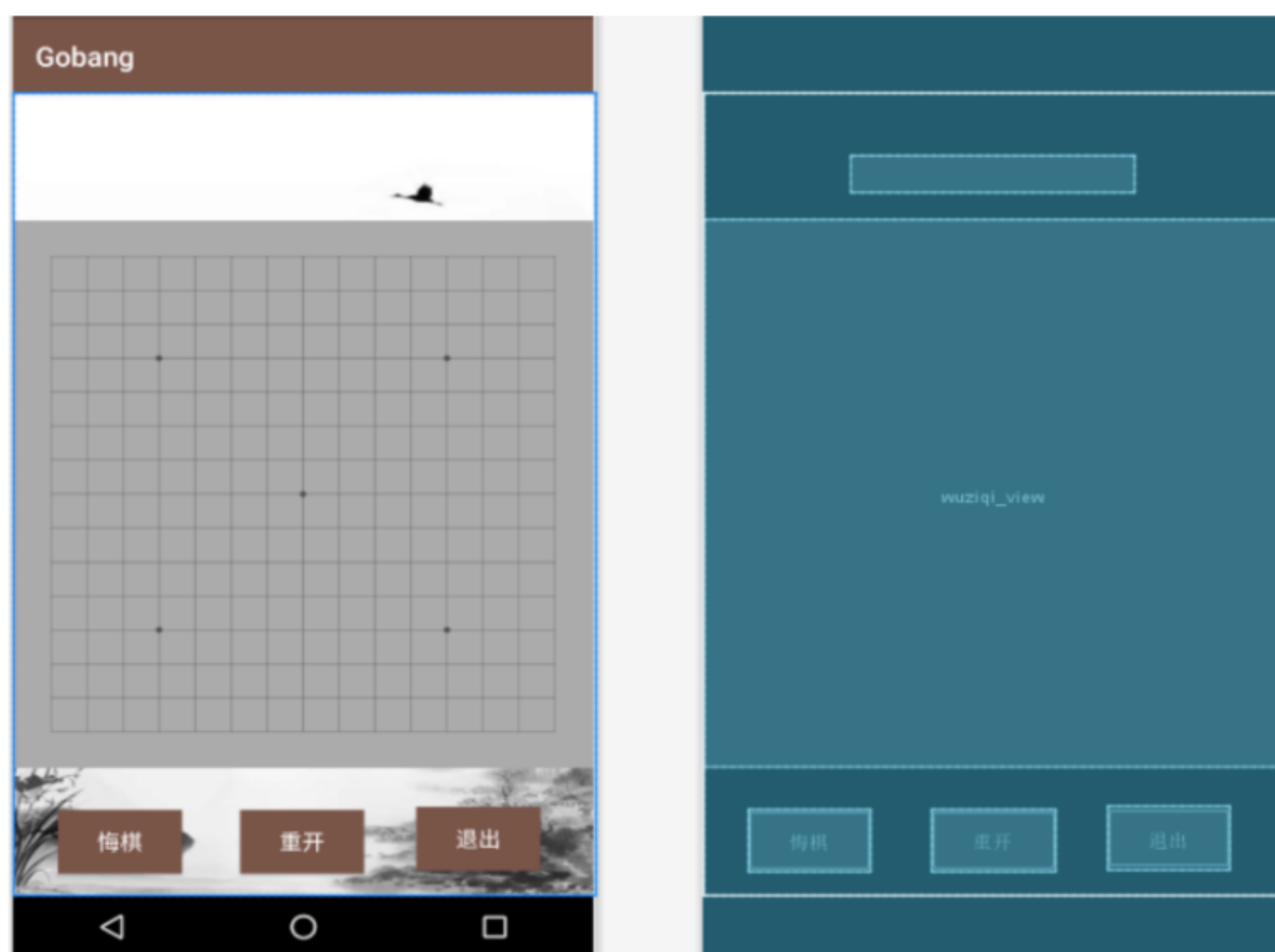
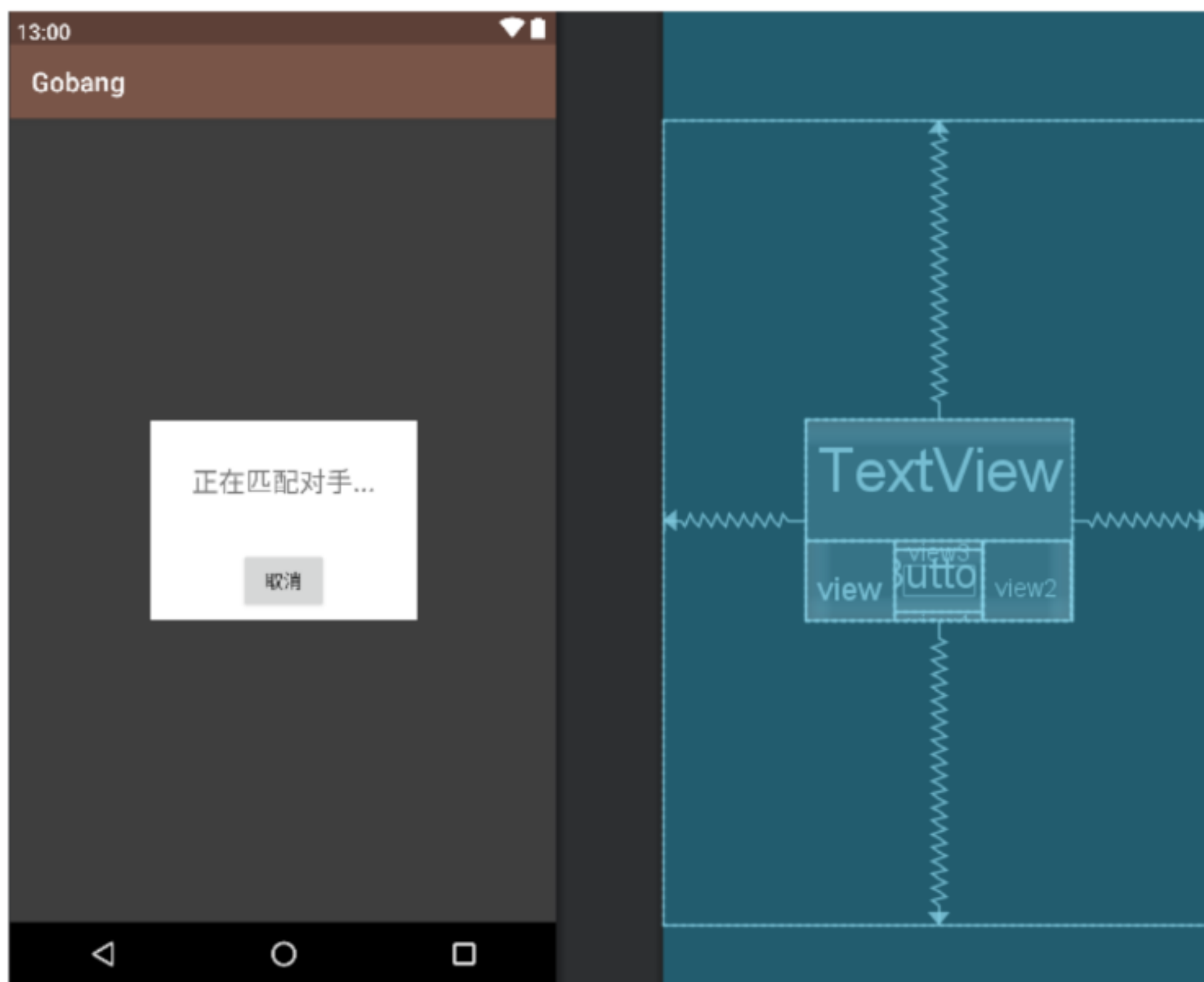
主页：



选择棋子界面：



游戏界面：



后端逻辑：

后端的 Java 层主页分为三大部分，分别是业务层、算法层、数据层。在业务层，主要是完成一些主体活动，例如注册、登录、开始游戏。这边开始游戏分为三种，一种是单人游戏，一种是双人对战、一种是联机对战。

下面看关键部分的代码。

设置点击事件来判断玩家按下的是哪个按钮：

```
public void onClick(View v) {
    JSONObject object = new JSONObject();
    switch (v.getId()) {
        case R.id.button1:
            startActivity(new Intent(this, MainActivity3.class));
            break;
        case R.id.button2:
            startActivity(new Intent(this, MainActivity2.class));
            break;
        case R.id.button3:
            if (!server_flag) {
                server_error();
                break;
            }
            startActivity(new Intent(this, MainActivity4.class));
            break;
        /*case R.id.button4:
            break;*/
        case R.id.login_button:
            initPop_login();
            break;
        case R.id.registered_button:
            initPop_registered();
            break;
        case R.id.login_confirm:
            login_1();
            break;
        case R.id.registered_confirm:
            registered_1();
            break;
        case R.id.login_cancel:
            logining_pop.dismiss();
            break;
        case R.id.registered_cancel:
```

```

        registered_pop.dismiss();
        break;}}
注册、登录方面核心代码:
boolean flag=uService.search(name);//方法封装在 uService 中, search 是搜索数据库, 判断是否已经存在用户名。
if(!psw_repeat.equals(psw)){

        Toast.makeText(this, "两次密码输入错误",
Toast.LENGTH_SHORT).show();
    }

    else if(flag){Toast.makeText(this, "注册失败,用户名已存在",
Toast.LENGTH_SHORT).show();}
    else {uService.register(user);

        Toast.makeText(MainActivity.this, "注册成功",
Toast.LENGTH_LONG).show();
        boolean item1=uService.registertime(name,str);
        boolean item2=uService.jifengone(name);
        registered_pop.dismiss();
    }
}

```

完成注册,同时完成各个表格的建立,数据的插入,例如单人积分表、登录时间表(这边默认注册时间为初始登录时间)、双人积分表等表格的建立汇总数据的插入,之后不再执行插入工作,而是执行数据更新工作。登录的代码也是如此,我将对数据库的操作封装到了一个类中,这个类既承担了数据库的初始建库工作,也保护了各个对数据库操作的方法,命名为 DBUtils.java。它包括

```

private DBOpenHelper dbHelper;
public DBUtils(Context context){
    dbHelper=new DBOpenHelper(context);
}

```

来调用 DBOpenHelper 实现数据库的初始化,也包含了很多方法,例如登录使要对数据库的操作的方法 public boolean login(String username,String password),又比如登录时要进行验证用户名是否存在的方法 public boolean search(String username)、public boolean register(User user)注册时插入数据时调用的方法、public boolean logintime(String username,String timed)记录登录时间并插入数据库的方法、public boolean registertime(String username,String timed)记录注册时间并插入数据库、public boolean jifengone(String username)注册时同步生成个人对战积分表时调用的插入数据库个人加分的方法、public boolean heqi(String username)和棋时对个人积分加分并更新到数据库时用到的方法、public boolean winqi(String username)赢棋时对个人积分加分并更新到数据库时用到的方法、public boolean writeqipu(String username,String qipud)比赛结束时打印棋盘记录到数据库的方法。这些方法都写在了一个类中,方便调用,也能重复

利用代码。

关于数据层：

首先，课设要求至少有五张表格，我采用的 sqlite 数据库，它是安卓自带的数据库，用起来方便，不需要再经过数据的转存等等麻烦的工序，不足之处时它没办法完成数据的通信，不同手机之间无法完成数据的交互。本次课设的一大特色是另外一个数据存储方式，实现了类似于云存储，将数据存储在了云服务器上。通过安卓 java 类 tcponline 来实现对固定 IP 地址端口号的访问，启动套字线程，向服务器发送请求。而服务器端，我用 IDEA 编写了一个程序，用与实现数据的中转服务、响应请求、服务本地数据库、返回数据，并且对对战双方进行线程控制。下面展示代码：

```
public boolean flag=true;
    @Override
    public void run() {
        super.run();
        Looper.prepare();
        while (flag) {
            try {
                final String string = bufferedReader.readLine();
                JSONObject object = JSONObject.parseObject(string);
                switch (object.getString("function")) {
                    case "join":
                        callback.init_chess(object);
                        break;
                    case "chess":
                        callback.rival_chess(object.getInteger("x"),
object.getInteger("y"));
                        break;
                    case "gameover":
                        flag=false;
                        callback.gameover(object);break;
                }
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
        Looper.loop();
    }
```

上面是安卓部分的代码，下面展示服务器代码：

```
public Server() {
    data = new player_data();
    mysql = new Mysql();

    System.out.println("服务器开始运行");

    try {
```



```

serverSocket = new ServerSocket(PORT); // 建立服务端，ip 为本机
ip,端口为 8886

Match_Thread match_thread = new Match_Thread(data);
match_thread.start();
while (true) {

    socket = serverSocket.accept(); // 监听客户端的连接，一旦有客
    户端连接，则会返回客户端对应的 accept

    Thread_Client thread_Client = new Thread_Client(socket, mysql,
data); // 启动线程

    thread_Client.start();

    System.out.println("已连接! "); }

} catch (IOException e) {
    e.printStackTrace();
} finally {
    try {
        serverSocket.close();
        socket.close();
    } catch (IOException e) {
        e.printStackTrace(); }
}}

```

关于算法层：

目前我所了解到的比较主流的是博弈树极大极小值 alpha-beta 剪枝搜索。基本思想就是采用启发式搜索、设定评估模型函数，进行打分。设定搜索深度，进行 alpha-beta 剪枝，抛弃不必要的计算和判断。

启发式搜索：

如果在标准的 15*15 棋盘中，要进行完全的计算，假设深度为 3（算三步），我们要进行的计算量是 225*224*223 种情况，这种计算量，哪怕对计算机来说，也是无比的巨大的，在棋盘没有摆满棋子之前，很多位子是不需要进行考虑的，为了尽量优先去计算那些有用的位子，所以有了启发式搜索模式，就是以最后一个落子为中心点，考虑它周围的八个落点，这算一层。

```

private boolean has_neighbors(Piece p) {

    //设定判断距离

    int s = 2;

    //防止溢出

    for (int i = (p.x - s > 0 ? p.x - s : 0); i <= p.x + s && i < 15; i++)

```

```

        for (int j = (p.y - s > 0 ? p.y - s : 0); j <= p.y + s && j < 15; j++)
            if (0 != gBoard[i][j])
                return true;
    return false;
}

```

这两个优化都是为了优先计算那些可能对局势优先影响的落子点，但是当这些都没有找到较好的模型时，这时候就没办法再优化了，只能逐个遍历那些还没计算的落子点，所以才会有最坏的情况 $225 \times 224 \times 223$ 这种情况。

启发式搜索之后需要进行对每个模型进行打分，安卓代码如下：

//计分板

```

private final int WIN5 = 100000; //连 5
private final int ALIVE4 = 10000; //活 4
private final int ALIVE3 = 1000; //活 3
private final int DIE4 = 1000; //冲 4
private final int ALIVE2 = 100; //活 2
private final int DIE3 = 100; //冲 3
private final int DIE2 = 10; //冲 2
private final int ALIVE1 = 10; //活 1

```

评分函数：

```

private int evaluate_point(Piece p, int chessColor) {
    int score = 0;
    int number, empty;
    int i, j;
    int x = p.x;
    int y = p.y;

    //横
    number = 0;
    empty = 0;
    for (i = x - 1; i >= 0 && i < x + 5; i++) {
        if (chessColor == gBoard[i][y])
            number++;
        else if (0 == gBoard[i][y]) {
            empty++;
            break;
        } else
            break;
    }
}

```

一共有四个方向，分别是横向获取、纵向获取、主斜线方向获取、反斜线方向获取。对四个方向获取棋子落子情况，并进行打分。

alpha-beta 剪枝优化：

对于计算机来说，这样的计算量，若是棋子数量不多的情况下还好，但随着棋子数量的增加，启发式搜索的数量越来越多在后面会机器每走一步，都需要大量的

时间来做运算，所以我们需要一个优化，来弥补随着棋子数量越来越多，前面启发式搜索的优化的失效带来的时间长的恶果。安卓代码如下：

```
private int alphabeta(int depth, int board[][], Piece p) {  
    int best = -WIN5;  
  
    ArrayList<Piece> option_queue = new ArrayList<Piece>(100); //待选的空
```

子队列

```
    ArrayList<Piece> sure_queue = new ArrayList<Piece>(); //最合适的下子
```

位置

```
    //更新棋盘
```

```
    for (int i = 0; i < 15; i++)  
        for (int j = 0; j < 15; j++)  
            gBoard[i][j] = board[i][j];  
    option_queue = generate_point();  
    //System.out.println(option_queue.size());  
    for (Piece i : option_queue) {  
        int x = i.x;  
        int y = i.y;  
        gBoard[x][y] = ComChessColor;  
        int tmp = min_alphabeta(depth - 1, i, -WIN5, WIN5);  
        if (tmp == best) {  
            sure_queue.add(i);  
        }  
        if (tmp > best) {  
            best = tmp;  
            sure_queue.clear();  
            sure_queue.add(i);  
        }  
        gBoard[x][y] = 0;  
    }  
}
```

```
    //如果有多个解 取随机
```

```
    Random random = new Random();  
    int k = random.nextInt(sure_queue.size());  
    p.x = sure_queue.get(k).x;  
    p.y = sure_queue.get(k).y;  
    return best;
```

```
}  
private int min_alphabeta(int depth, Piece p, int alpha, int beta) {
```



```
int res = evaluate_situation();

//代表临时 beta

int best = WIN5;
ArrayList<Piece> v = new ArrayList<>(50);

//判断是否胜利

if (depth <= 0 || p.is_win(gBoard, RivalChessColor))
    return res;

//生成待搜索序列

v = generate_point();
for (Piece i : v) {
    int x = i.x;
    int y = i.y;
    gBoard[x][y] = RivalChessColor;
    int tmp = max_alphabeta(depth - 1, p, alpha, Math.min(best, beta));
    gBoard[x][y] = 0;
    best = Math.min(tmp, best);
    if (tmp < alpha)
        break;
}
return best; }
```

四、运行结果展示

进入安卓应用：



注册：



登录：



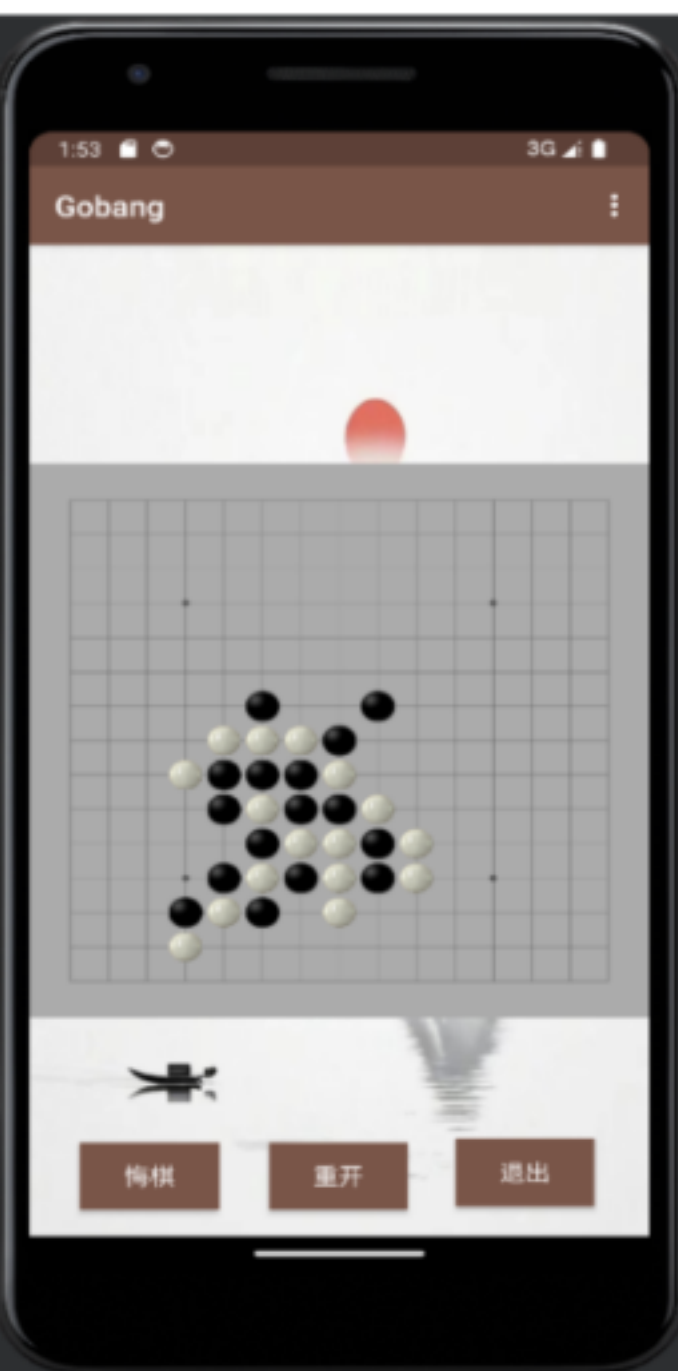
登陆成功：



单人游戏：选择棋子



开始对战：



课程设计心得

在进行本次课程设计之前，我想了很多方面的问题，在大学生时代设计安卓软件时，首先是不知设计哪方面的软件以此来满足市场的需求。在看了很多软件设计，加上自己可以完成的。最终选择了编写一个五子棋博弈软件，之所以选择编写这个五子棋软件，一方面是因为我之前参加过相应的比赛，学习过相应的算法，做起来比较得心应手。另外一方面，五子棋博弈其实还有一个方面一直是我比赛时没接触到的，而又是未来计算发展的必然需求，那就是网络化交互。之前的比赛都是本地运行程序，使用者充当信息交互中间商。但是现在放眼望去，基本上所有的安卓 APP 基本上都有数据交互这一功能，甚至将相关的 APP 数据存储在了云端。而五子棋博弈软件又是一个信息交互最为频繁的一种游戏，正好可以给我联系然后进行数据交互，数据存储等等操作。这也算是一个小突破，因为以前的其他课的课设基本上都是本地上运行，哪怕布置到了 web，基本上也只能是本机访问。一方面是电脑的限制，个人使用的电脑有内网限制，需要做内网穿透，自己的电脑的 IP 端口号才可以被其他电脑访问。一方面是技术的选择，我选择了云服务器，来替代本地电脑运行服务器程序，进行数据的交互管理。使用 IDEA 编写线程控制程序，并用 Maven 将相关程序打包成 jar，在我的云服务器上安装 java 部署器，将 jar 包上传到云服务器上，并且在云服务器上建立相关的 MySQL 数据库，以 tomcat 来运行 jar 包。而且云服务器还有一个好处，那就是，它可以全天 24 小时运行程序，从不间断。这样一来，就实现了五子棋棋手可以在网上寻找自己的对手进行对战，不用空间、设备的限制了。

未来要优化的方向，一个方面是数据库的表格要多建立几个，直至保证和 sqlite 的本地数据库表格一样多。第二个优化方向是联机对战可以加入随机匹配算法，不再单单局限于只能两个选择联机对战就匹配成对手，而是可以根据每个棋手的积分来匹配对手。